

SCO Approach to k-MDB

Kiyaan Pillai

July 2026

1 Introduction

Cui et al. (2025) and Wang et al. (2025) proposed a branch-and-bound algorithm with several novel optimizations. They both propose heuristic approaches to identifying maximum k -MDB structures. However, these approaches are intended as a cheap approximation to tighten the upper bound that their pruning optimizations rely on. Thus, these heuristics often fail to find large k -MDB on a graph.

The proposed branch algorithms run in exponential time. While benchmark tests often run on graphs of millions of edges, practical applications might need to run on graphs of billions of edges. For instance, Cui et al.’s proposed branch-and-pivot (BnP) algorithm took up to 2 hours and 25 minutes on a graph with 20,592,962 edges. A heuristic that runs in linear time of n and m on a graph with n nodes and m edges, and is more accurate than the previously proposed algorithms, is required.

Additionally, the proposed heuristics’ accuracy is heavily dependent on k . We observe that on a graph whose maximum solution has k_a missing edges, previous heuristics fail to find it more often when $k > k_a$.

Finally, the proposed heuristics’ accuracy is heavily dependent on θ as a parameter. θ is intended to be a

We propose a novel adaptation of Ant Colony Optimization that runs in linear time of n and m , is parallelizable, and whose accuracy is significantly higher than the previous proposed heuristics, rivals optimal algorithms, and whose runtime is significantly less dependent on θ and k .

2 Definitions

Define a bipartite graph as a graph $G = (U, V, E)$, where U and V are two disjoint sets of vertices, and $E \subseteq U \times V$ is the set of edges between U and V . Let $S = (U_S, V_S)$ denote a subset of vertices where $U_S \subseteq U$ and $V_S \subseteq V$. Let $G(S) = (U_S, V_S, E_S)$ be the induced subgraph of S where $E_S = \{(u, v) \mid u \in U_S \wedge v \in V_S \wedge (u, v) \in E\}$. Define $\bar{E}(S) = \{(u, v) \mid u \in U_S \wedge v \in V_S \wedge (u, v) \notin E\}$ be the set of non-edges in a subgraph. Let a k -defective biclique be a subgraph S where $|\bar{E}(S)| \leq k$.

Define the neighbors of a node u in a subgraph S as $N_S(u) = \{v \mid \{u, v\} \in E(S)\}$. Define the nonneighbors of a node as $\bar{N}_S(u) = \{v \mid (u, v) \notin E(S)\}$. Define the degree of a node as $d_S(u) = |N_S(u)|$, and the nondegree as $\bar{d}_S(u) = |\bar{N}_S(u)|$.

Define expanding a k -MDB S as adding a vertex u such that $|\bar{E}(S \cup \{u\})| \leq k$. Define a maximal k -MDB as a subgraph S that cannot be expanded. That is, for any node $u \notin S$, $|\bar{E}(S \cup \{u\})| > k$. Define the maximum k -MDB as the largest maximal k -MDB in a graph G .

3 Methods

We propose Slime Colony Optimization (SCO), an adaptation of ACO fitted to biclique graphs. Pheromones allow agents to converge on optimal solutions, and subspecies of slimes allow for multiple near-optimal solutions.

Pseudocode:

Have to update this to ablate the adding pheromone to the best solutions
Will then have to include the instance fitness and everything I tried there
Also have to try different desirability functions
Could I genetically evolve the desirability function itself?

Algorithm 1 SCO

Ensure: a subset of vertices S such that $G(S)$ is a k -MDB

```

function SCO( $G, N_e, N_g, E, p$ )
  for all  $u \in G$  do
     $P(u) \leftarrow 1$ 
   $e \leftarrow 0$ 
  while  $e < N_e$  do
     $e \leftarrow e + 1$ 
     $A \leftarrow N_a$  empty subgraphs
     $P_C \leftarrow P$ 
    parallel for each  $S_i \in S$  do
       $next \leftarrow \text{softmax}_{u \in G \setminus S}(\text{DESIRABILITY}(u, S_i, P_C))$ 
      add  $next$  to  $S_i$ 
       $P_C(u) \leftarrow P_C(u) + p$ 
     $P \leftarrow P_C$ 
    for all  $u \in G$  do
       $P(u) \leftarrow P(u) \times E$ 
  for all  $u \in G$  do
     $P(u) \leftarrow P(u) \times E$ 
  return  $T$ 
function DESIRABILITY( $u, S, P$ )
  return  $P(u)^3 \times d_S(u)$ 

```

We also included a genetic algorithm to benchmark against ACO:

Algorithm 2 Genetic Algorithm

Ensure: a subset of vertices S such that $G(S)$ is a k -MDB

```
function GENETICALGORITHM( $G, N_G, N_e, m_k$ )
   $P \leftarrow N_i$  distinct subgraphs  $\{\text{softmax}_{u \in G}(d(u))\}$ 
   $K \leftarrow k$ 
   $e \leftarrow 0$ 
   $P_n \leftarrow P$ 
   $K_N \leftarrow K$ 
  while  $e < N_e$  do
    parallel for each  $i \in [1, |P|]$  do
       $m, f \leftarrow 2$  distinct  $\text{argsoftmax}_{P_C \in P \setminus P_i}(\text{INSTANCEFITNESS}(P_C))$ 
       $k_O \leftarrow \text{CROSSOVERK}(K_m, K_f)$ 
       $(P_n)_i \leftarrow \text{CROSSOVERS}(P_m, P_f, k_O)$ 
       $(K_N)_i \leftarrow k_O$ 
function CROSSOVERK( $K_m, K_f$ )
  if  $\text{uniform}(0, 1) \leq 0.5$  then
     $k_O \leftarrow K_m$ 
  else
     $k_O \leftarrow K_f$ 
  if  $\text{uniform}(0, 1) \leq m_k$  then
    if  $\text{uniform}(0, 1) \leq 0.5$  then
       $k_O \leftarrow \max(0, K_O - 1)$ 
    else
       $k_O \leftarrow \min(k, K_O + 1)$ 
  return  $k_O$ 
function CROSSOVERS( $S_m, S_f, k_O$ )
   $C \leftarrow S_m \cap S_f$ 
  if  $|C| = 0$  then
     $U \leftarrow S_m \cup S_f$ 
     $C \leftarrow \{\}$ 
    while  $|E(\bar{C})| < k$  and  $|U| > 0$  do
       $u \leftarrow \text{argmax}_{v \in C} d_c(v)$ 
       $C \leftarrow C \cup \{u\}$ 
       $U \leftarrow U \setminus \{u\}$ 
  while  $|\bar{E}(C)| > k$  do
     $C \leftarrow C \setminus \{\text{argmax}_{u \in C} \bar{d}_C(u)\}$ 
  REPAIR( $C$ )
  return  $C$ 
function INSTANCEFITNESS( $S = (U_S, V_S)$ )
   $m \leftarrow 1$ 
  if  $\min(|U_S|, |V_S|) \geq \theta$  then
     $m \leftarrow \max |U_S, V_S|$ 
  return  $m \times \min(|U_S|, |V_S|)^2$ 
```

Algorithm 3 Tabu Repair

function TABUREPAIR(G, S, tt, p) $p_A \leftarrow 0$ **for all** $u \in G$ **do** $T(u, \text{add}) = 0$ $T(u, \text{remove}) = 0$ $F_B \leftarrow \text{INSTANCEFITNESS}(S)$ $S_B \leftarrow S$ **while** $p_A \leq p$ **do** $T_C \leftarrow T$ $F_C \leftarrow \text{INSTANCEFITNESS}(S)$ $F_O, S_O \leftarrow -\infty, S$ $E \leftarrow \{(u, \text{add}) \in G \setminus S\} \cup \{(u, \text{remove}) \in S\}$ **for** $(u, \text{add}) \in E$ s.t. $T_C(u, \text{add}) = 0$ **do****if** add **then** $S_N \leftarrow S \cup \{u\}$ **else** $S_N \leftarrow S \setminus \{u\}$ $F_N \leftarrow \text{INSTANCEFITNESS}(S_N)$ **if** $F_N < F_C$ **then** $T_C(u, \text{add}) \leftarrow tt$ **if** $F_N > F_O$ **then** $F_O, S_O \leftarrow F_N, S_N$ **if** $F_O > F_B$ **then** $F_B \leftarrow F_O$ $S_B \leftarrow S_O$ **for** $(u, \text{add}) \in E$ s.t. $T_C(u, \text{add}) > 0$ **do****if** add **then** $S_N \leftarrow S \cup \{u\}$ **else** $S_N \leftarrow S \setminus \{u\}$ $F_N \leftarrow \text{INSTANCEFITNESS}(S_N)$ **if** $F_N > F_B$ **then** $F_B, S_B \leftarrow F_N, S_N$ $T \leftarrow T_C$ **return** S_B

NEXT: Figure out what datasets I want to compare against, implement subspecies, and then write the pseudocode